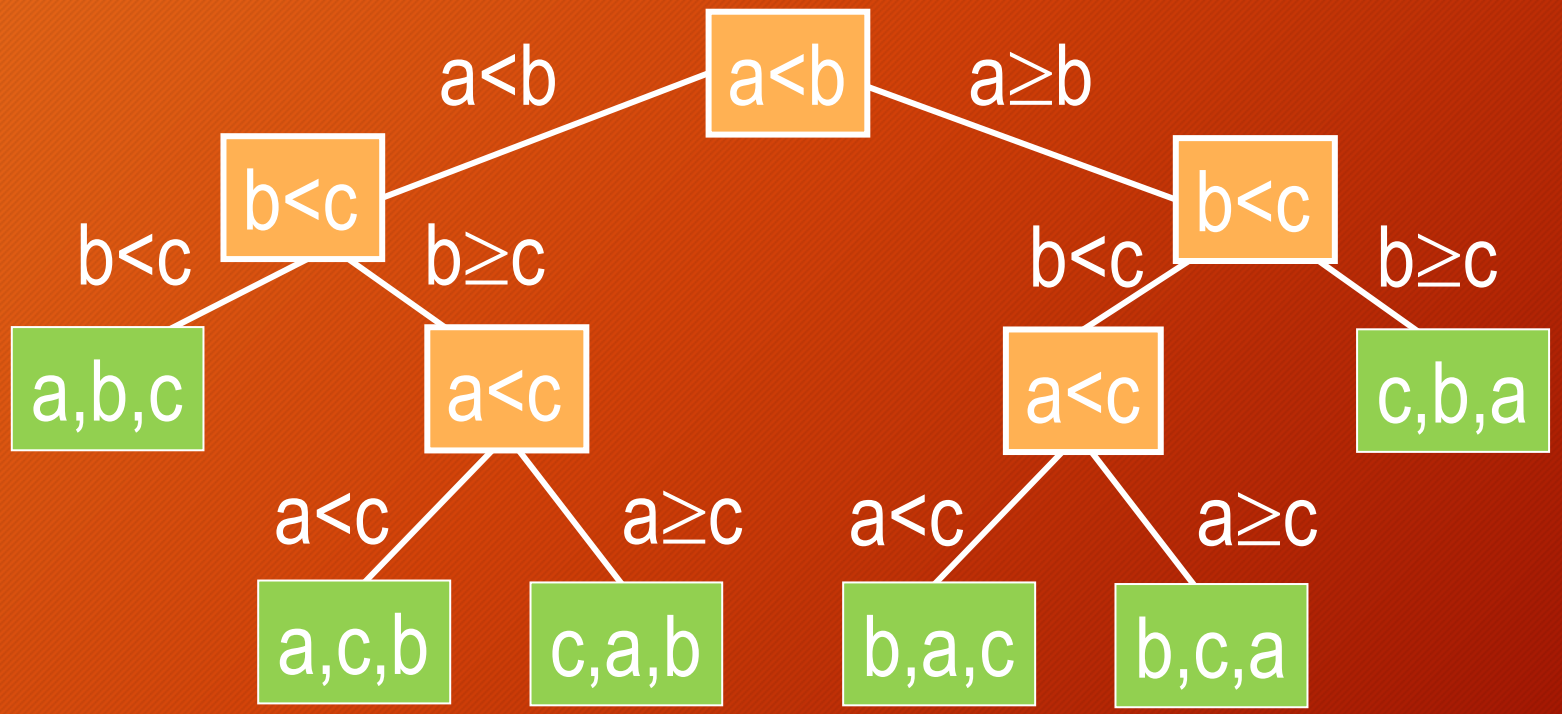
Ordenação em Tempo Linear

Algoritmos baseados em comparação

- **Teorema.** Qualquer algoritmo de ordenação por comparação exige $\Omega(n \log n)$ comparações no pior caso.

Algoritmos baseados em comparação

- Exemplo: Ordenar três números inteiros a, b e c (Árvore de Decisão)



Algoritmos baseados em comparação

- A **complexidade de pior caso** é a **maior profundidade** de uma folha **=** altura da árvore.
- Relação entre o número de folhas e a altura de uma árvore binária completa é **folhas = 2^{altura}**
- Número de **folhas = *n*!,** onde *n* é o número de elementos a serem ordenados.

Algoritmos baseados em comparação

- Como a árvore de decisão não é necessariamente completa temos que:

$$n! \leq 2^{\text{altura}} \Rightarrow \log_2 n! \leq \log_2 2^{\text{altura}} \Rightarrow \text{altura} \geq \lceil \log_2 n! \rceil$$

- Aproximação de Stirling

$$\lceil \log_2 n! \rceil = n \log_2 n - \frac{n}{\ln 2} + \frac{1}{2} \log_2 n + O(1)$$

Ordenação Linear

Vamos ver alguns algoritmos que em certas condições podem fazer a ordenação em $O(n)$.



Ordenação por Contagem

- Pressupõe que cada um dos n elementos de entrada é um inteiro no intervalo de 1 a k, para algum inteiro k. Quando k é $O(n)$, a ordenação é executada em $\theta(n)$.

Ordenação por Contagem

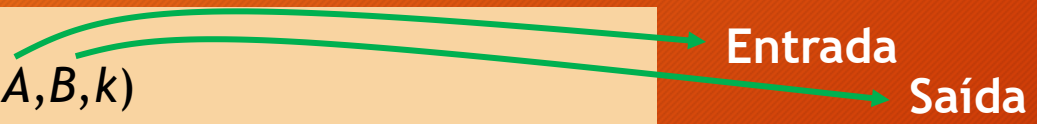
- Pressupõe que cada um dos n elementos de entrada é um inteiro no intervalo de 1 a k, para algum inteiro k. Quando k é $O(n)$, a ordenação é executada em $\theta(n)$.

- Ideia Básica: Dado um elemento x determinar quantos elementos são menores que x. Essa informação é utilizada para inserir x diretamente na sua posição na lista ordenada.

Ordenação por Contagem

```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```



Ordenação por Contagem

```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```

Inicialização do contador

Conta o número de elementos *x*

Conta o número de elementos menores que *x*

Ordenação por Contagem

```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```

Monta a saída

Atualiza o contador

Ordenação por Contagem

	1	2	3	4	5	6	7	8
A	3	0	5	2	5	0	5	3

Ordena_cont(*A*,*B*,*k*)

para *i* ← 0 até *k* faça
    C[*i*] ← 0
para *j* ← 1 até *comprimento_A* faça
    C[*A*[*j*]] ← *C*[*A*[*j*]] + 1
para *i* ← 1 até *k* faça
    C[*i*] ← *C*[*i*] + *C*[*i*-1]
para *j* ← *comprimento_A* até 1 faça
    B[*C*[*A*[*j*]]] ← *A*[*j*]
    C[*A*[*j*]] ← *C*[*A*[*j*]] -1

Ordenação por Contagem

	1	2	3	4	5	6	7	8
A	3	0	5	2	5	0	5	3

	0	1	2	3	4	5
C	2	0	1	2	0	3



```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```

Ordenação por Contagem

	1	2	3	4	5	6	7	8
A	3	0	5	2	5	0	5	3

	0	1	2	3	4	5
C	2	0	1	2	0	3

	0	1	2	3	4	5
C	2	2	3	5	5	8



```
Ordena_cont(A,B,k)

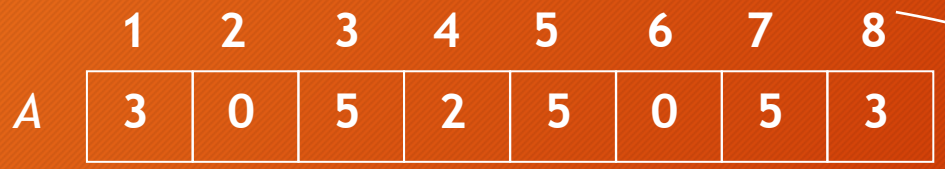
para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```

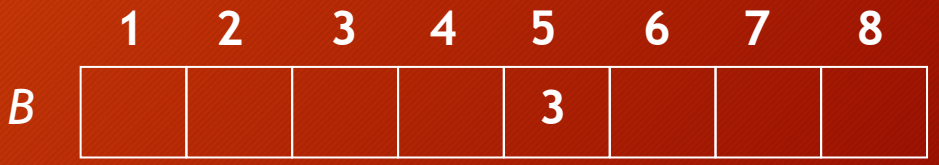
Ordenação por Contagem

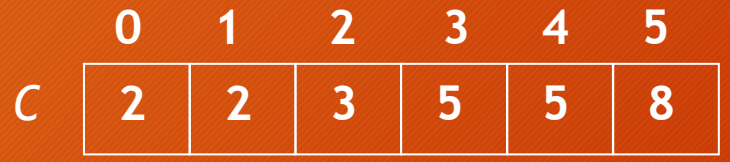
Ordena_cont(*A*,*B*,*k*)

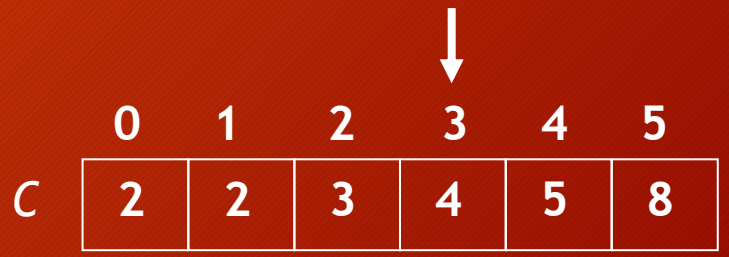
...
para *j* ← *comprimento_A* até 1 faça
    B[*C*[*A*[*j*]]] ← *A*[*j*]
    C[*A*[*j*]] ← *C*[*A*[*j*]] -1

j = 8
A[*j*] = 3
C[*A*[*j*]] = 5





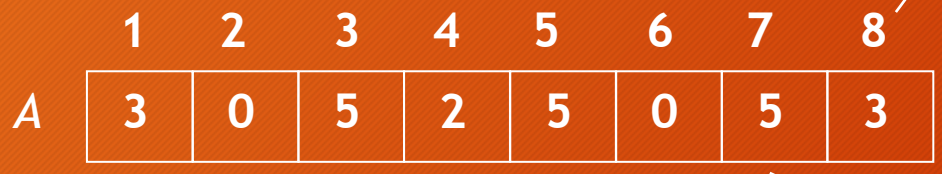


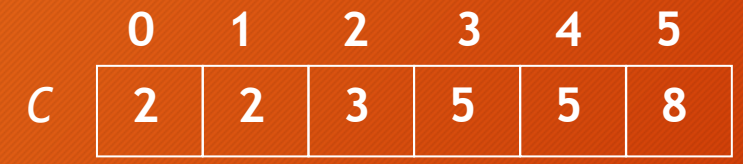


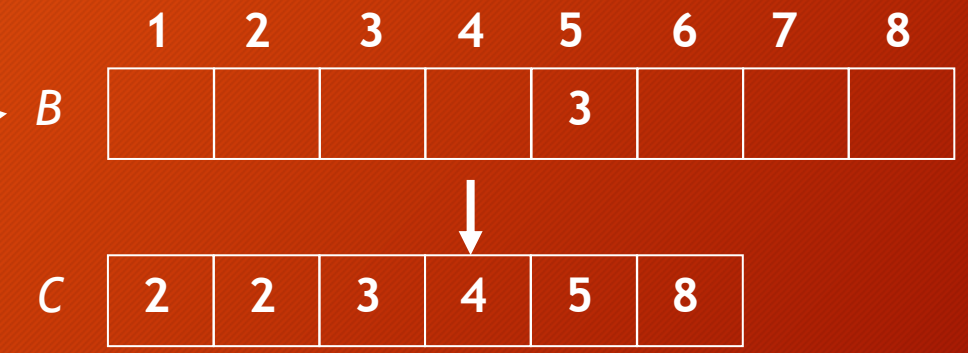
Ordenação por Contagem

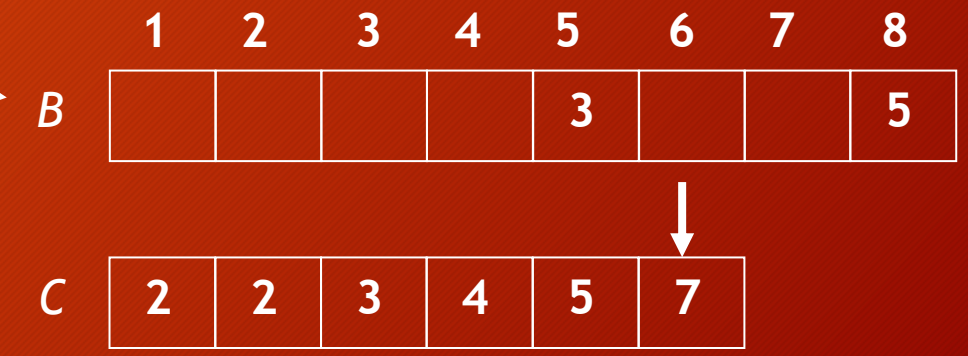
```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```





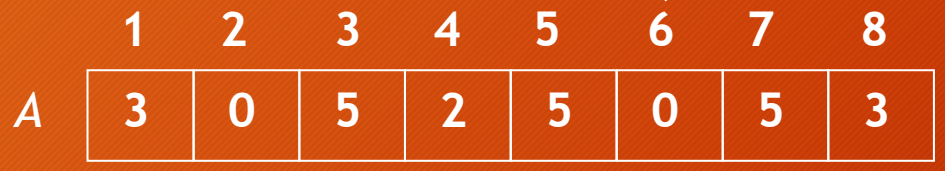


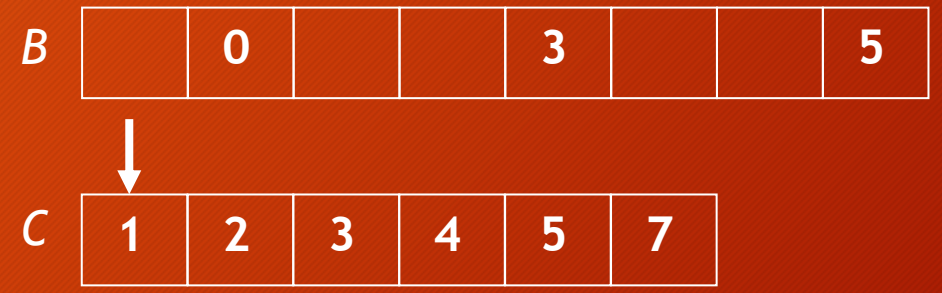


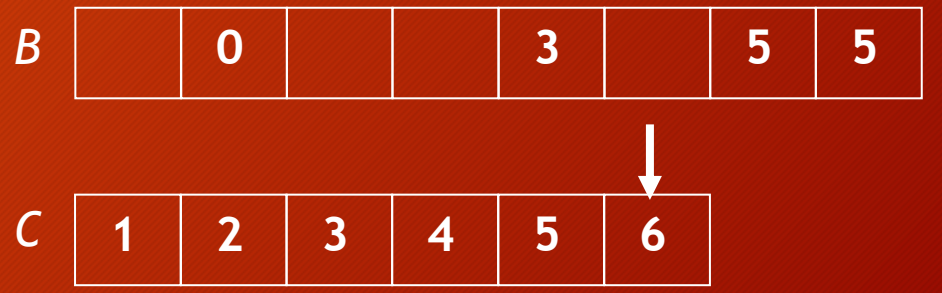
Ordenação por Contagem

```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```







Ordenação por Contagem

```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```

	1	2	3	4	5	6	7	8
A	3	0	5	2	5	0	5	3

B		0	2		3		5	5

C	1	2	2	4	5	6

B	0	0	2	3	3	5	5	5

Complexidade Ordenação por Contagem

- Primeiro laço : $\theta(k)$

```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```

Complexidade Ordenação por Contagem

- Primeiro laço : $\theta(k)$
- Segundo laço : $\theta(n)$

```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```

Complexidade Ordenação por Contagem

- Primeiro laço : $\theta(k)$
- Segundo laço : $\theta(n)$
- Terceiro laço : $\theta(k)$

```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```

Complexidade Ordenação por Contagem

- Primeiro laço : $\theta(k)$
- Segundo laço : $\theta(n)$
- Terceiro laço : $\theta(k)$
- Quarto laço : $\theta(n)$
- Total: $\theta(k+n)$

Se k é $O(n)$, então Ordenação por Contagem é $\theta(n)$

```
Ordena_cont(A,B,k)

para i ← 0 até k faça
    C[i] ← 0
para j ← 1 até comprimento_A faça
    C[A[j]] ← C[A[j]]  + 1
para i ← 1 até k faça
    C[i] ← C[i] + C[i-1]
para j ← comprimento_A até 1 faça
    B[C[A[j]]] ← A[j]
    C[A[j]] ← C[A[j]] -1
```

Complexidade Ordenação por Contagem

- Propriedade Importante:

Esse método é ESTÁVEL

→ Números com o mesmo valor aparecem na lista de saída na mesma ordem em que se apresentam na lista de entrada.

Radix Sort

- Considere *n* números com *d* dígitos cada

Radix Sort

- Considere *n* números com *d* dígitos cada
- Imagine cada número em uma linha e cada dígito em uma coluna

Radix Sort

- Considere *n* números com *d* dígitos cada
- Imagine cada número em uma linha e cada dígito em uma coluna
- Radix Sort verifica cada coluna, individualmente, começando do dígito menos significativo
-

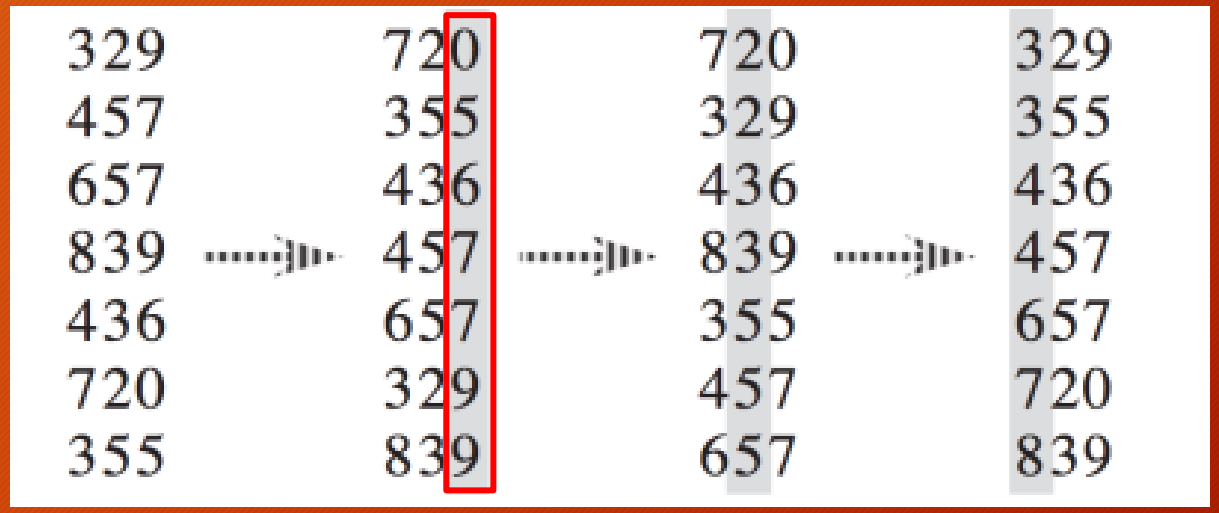
Radix Sort

- Considere *n* números com *d* dígitos cada
- Imagine cada número em uma linha e cada dígito em uma coluna
- Radix Sort verifica cada coluna, individualmente, começando do dígito menos significativo
- Para cada coluna, o algoritmo ordena os dígitos pertencentes a tal coluna

Radix Sort

- Exemplo

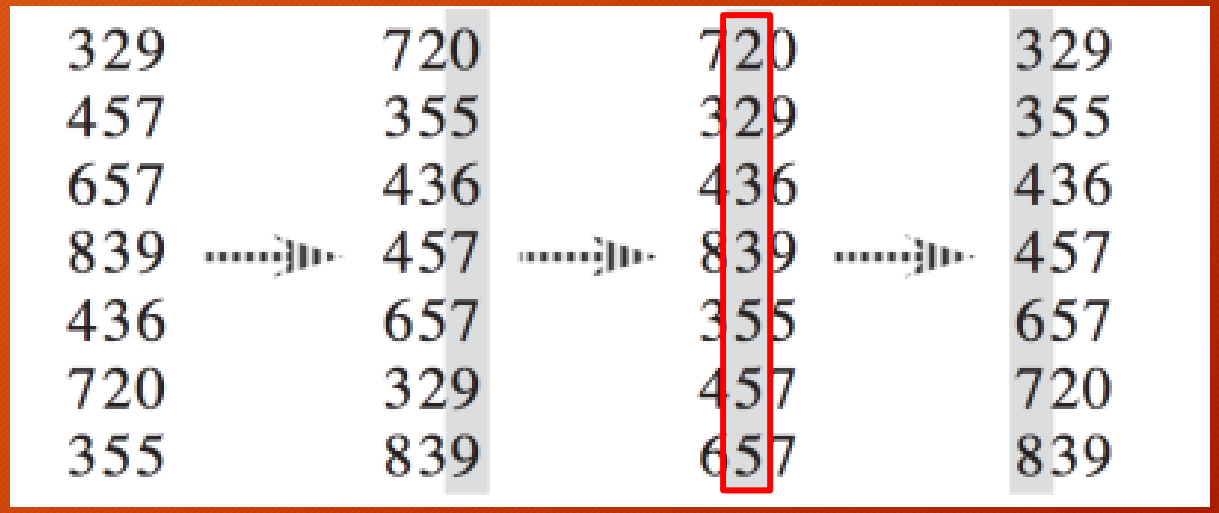
  Lista a ser ordenada: 329, 457, 657, 839, 436, 720, 355



Radix Sort

- Exemplo

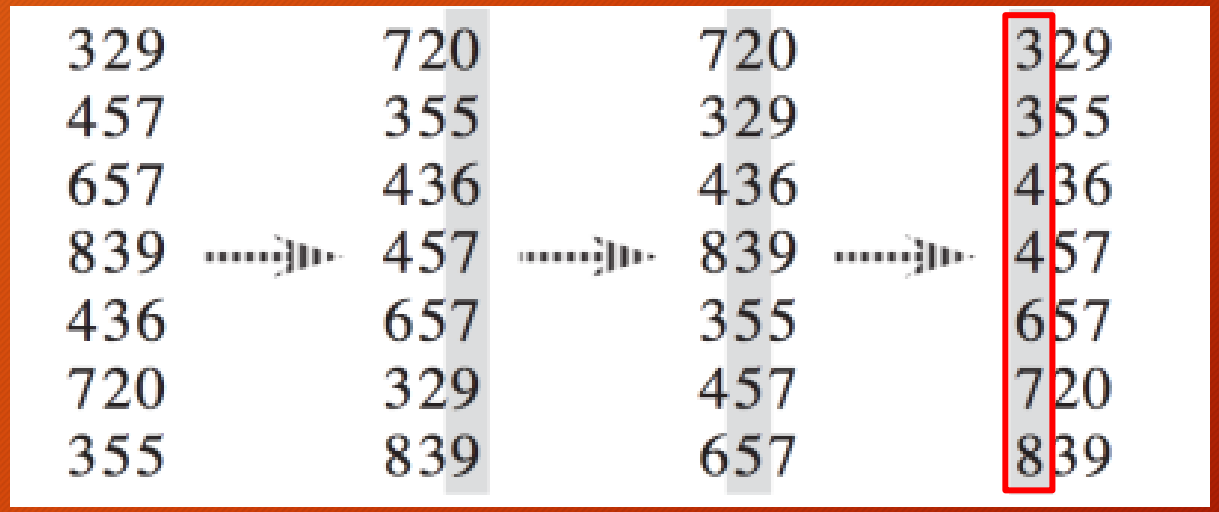
Lista a ser ordenada: 329, 457, 657, 839, 436, 720, 355



Radix Sort

- Exemplo

  Lista a ser ordenada: 329, 457, 657, 839, 436, 720, 355



- O algoritmo de ordenação usado em cada coluna deve ser Estável

Radix Sort

Radix_sort(*A*,*d*)

para *i* ← 1 até *d* faça
    use um algoritmo Estável para ordenar A no *i*-ésimo dígito

Radix Sort – Prova de correção

Radix_sort(*A*,*d*)

para *i* ← 1 até *d* faça
    use um algoritmo Estável para ordenar A no *i*-ésimo dígito

Invariante do Laço
No início da linha 1, a lista está ordenada para *i*-1 dígitos menos significativos.

Radix Sort – Prova de correção

Radix_sort(*A*,*d*)

para *i* ← 1 até *d* faça
    use um algoritmo Estável para ordenar A no *i*-ésimo dígito

Invariante do Laço
No início da linha 1, a lista está ordenada para *i*-1 dígitos menos significativos.

Início
A lista está trivialmente ordenada para 0 dígitos.

Radix Sort – Prova de correção

Radix_sort(*A*,*d*)

para *i* ← 1 até *d* faça
    use um algoritmo Estável para ordenar A no *i*-ésimo dígito

Passo indutivo
Assuma que a lista está ordenada para *i*-1 dígitos menos significativos.
Se os dígitos da coluna *i* são distintos, é trivial que os números serão ordenados corretamente.
Se há dois dígitos iguais na coluna *i*, ainda assim a ordenação está correta, porque o método usado é estável, ou seja, mantém os dígitos iguais na mesma ordem de entrada (que por hipótese estava ordenada para *i*-1 dígitos.

Radix Sort – Prova de correção

Radix_sort(*A*,*d*)

para *i* ← 1 até *d* faça
    use um algoritmo Estável para ordenar A no *i*-ésimo dígito

Passo indutivo
Assuma que a lista está ordenada para *i*-1 dígitos menos significativos.
Se os dígitos da coluna *i* são distintos, é trivial que os números serão ordenados corretamente.
Se há dois dígitos iguais na coluna *i*, ainda assim a ordenação está correta, porque o método usado é estável, ou seja, mantém os dígitos iguais na mesma ordem de entrada (que por hipótese estava ordenada para *i*-1 dígitos.

Término
O laço termina quando $i = d + 1$.

Radix Sort - Complexidade

- Sendo n números e d dígitos, onde cada dígito pode assumir k valores
- Se a ordenação for $\Theta(n+k)$, então o Radix Sort é $\Theta(d(n+k))$.
- Quando d é constante e k é $\Theta(n)$, então o Radix Sort é $\Theta(n)$.

```
Radix_sort(A,d)

para i ← 1 até d faça
    use um algoritmo Estável para ordenar A no i-ésimo dígito
```

Ordenação por Baldes (Bucket_sort)

- Tempo linear quando a entrada é gerada a partir de uma distribuição uniforme.

- Pressupõe que os elementos se distribuem uniformemente no intervalo [0,1).

Ordenação por Baldes (Bucket_sort)

- Tempo linear quando a entrada é gerada a partir de uma distribuição uniforme.
- Pressupõe que os elementos se distribuem uniformemente no intervalo [0,1).
- Divide o intervalo [0,1) em *n* subintervalos de igual tamanho, ou *baldes*, e depois distribui os *n* elementos da entrada entre os baldes.
- Como a distribuição é uniforme, não se espera que muitos elementos caiam no mesmo balde.
- O algoritmo faz uso de um vetor auxiliar *B* que mantém as listas de cada balde.

Ordenação por Baldes (Bucket_sort)

ordena_balde(*A*)

n ← *comprimento_A*

para *i* ← 1 até *n* faça
    inserir *A*[*i*] na lista *B*[⌊*nA*[*i*]⌋]

para *i* ← 0 até *n*-1 faça
    ordenar lista *B*[i] com Ordenação por Inserção

concatenar as listas *B*[0], ..., *B*[*n*-1] em ordem

Ordenação por Baldes (Bucket_sort)

```
ordena_balde(A)

n ← comprimento_A

para i ← 1 até n faça
    inserir A[i] na lista B[⌊nA[i]⌋]

para i ← 0 até n-1 faça
    ordenar lista B[i] com Ordenação por Inserção

concatenar as listas B[0], ..., B[n-1] em ordem
```

	A
1	0,21
2	0,37
3	0,94
4	0,52
5	0,29
6	0,72
7	0,25
8	0,06
9	0,31
10	0,78

Ordenação por Baldes (Bucket_sort)

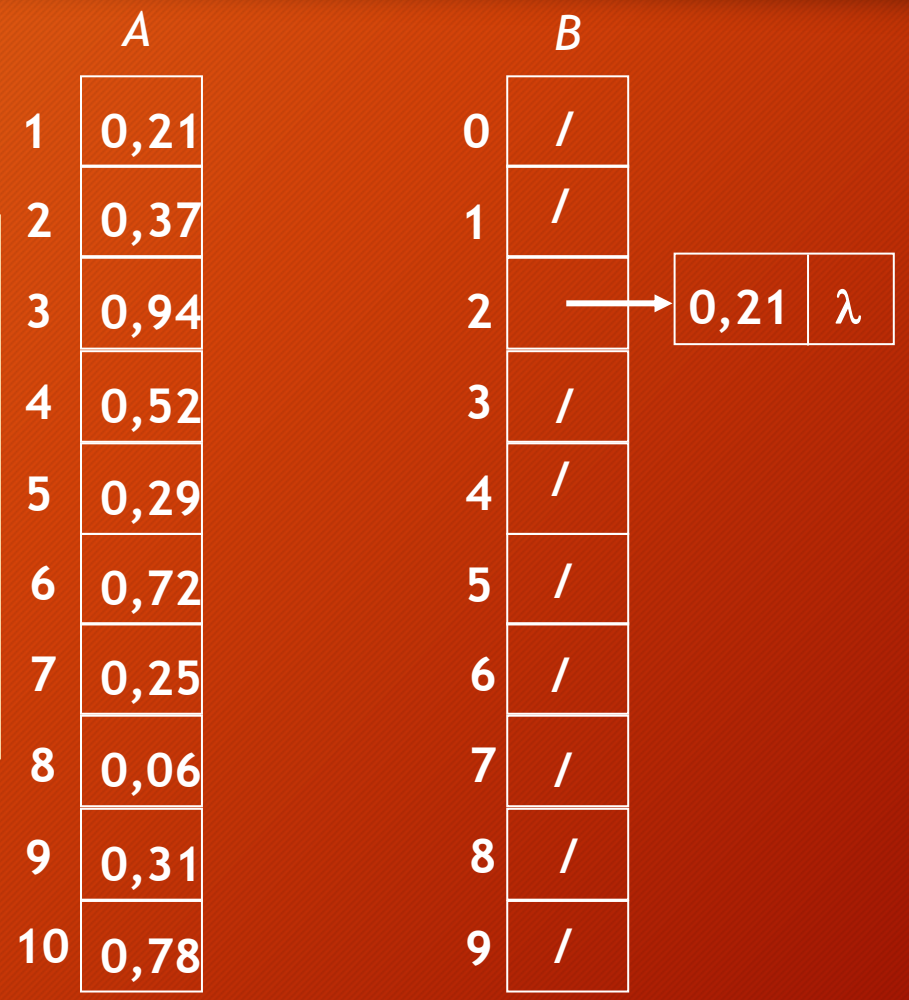
```
ordena_balde(A)

n ← comprimento_A

para i ← 1 até n faça
    inserir A[i] na lista B[⌊nA[i]⌋]

para i ← 0 até n-1 faça
    ordenar lista B[i] com Ordenação por Inserção

concatenar as listas B[0], ..., B[n-1] em ordem
```



Ordenação por Baldes (Bucket_sort)

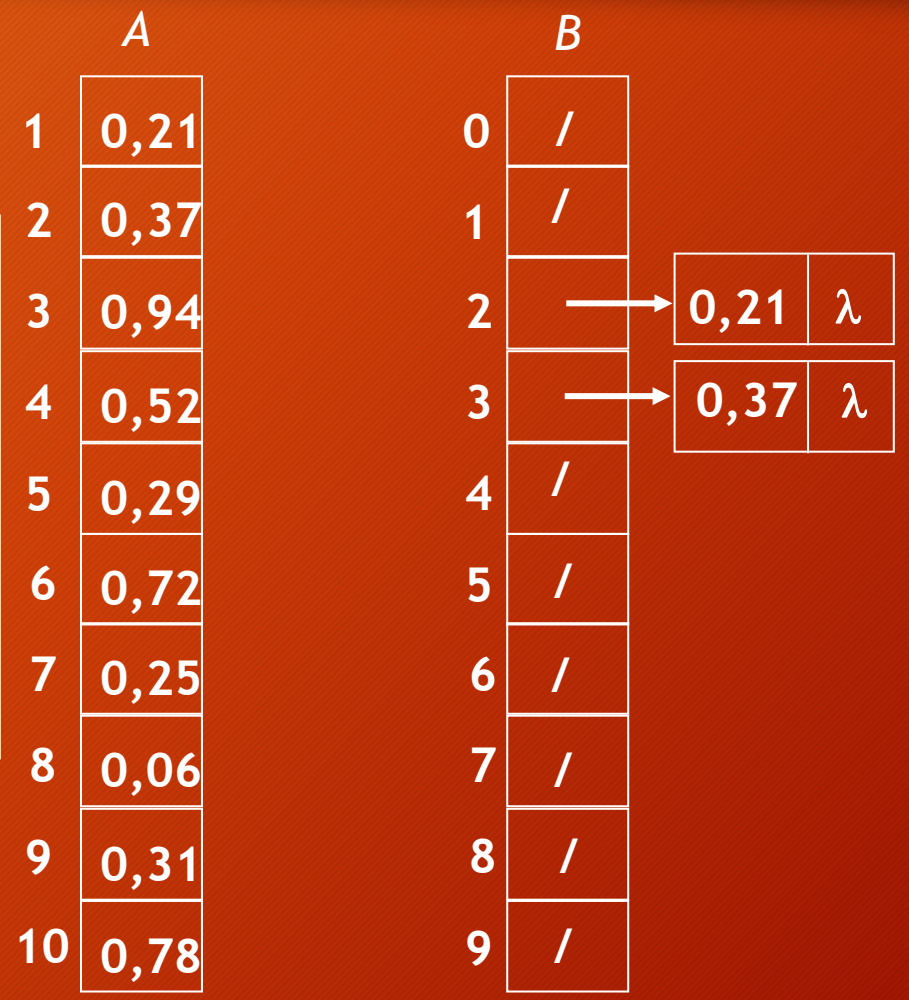
```
ordena_balde(A)

n ← comprimento_A

para i ← 1 até n faça
    inserir A[i] na lista B[⌊nA[i]⌋]

para i ← 0 até n-1 faça
    ordenar lista B[i] com Ordenação por Inserção

concatenar as listas B[0], ..., B[n-1] em ordem
```



Ordenação por Baldes (Bucket_sort)

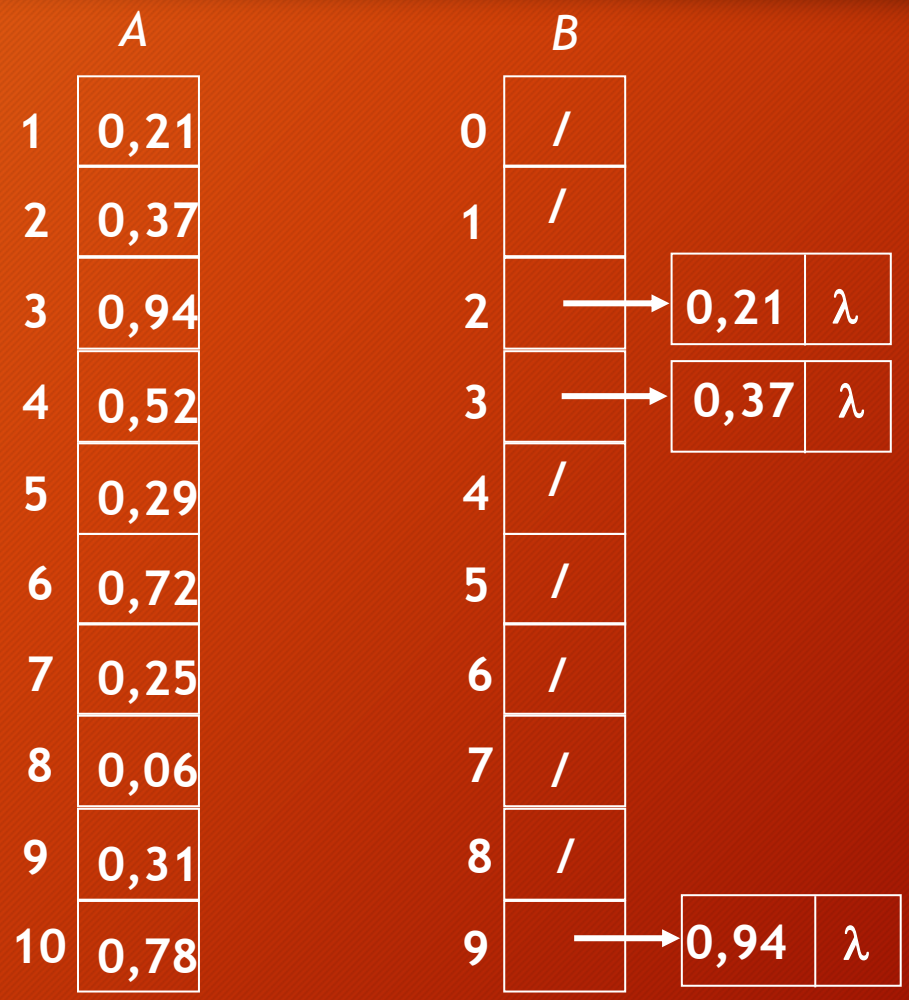
```
ordena_balde(A)

n ← comprimento_A

para i ← 1 até n faça
    inserir A[i] na lista B[⌊nA[i]⌋]

para i ← 0 até n-1 faça
    ordenar lista B[i] com Ordenação por Inserção

concatenar as listas B[0], ..., B[n-1] em ordem
```



Ordenação por Baldes (Bucket_sort)

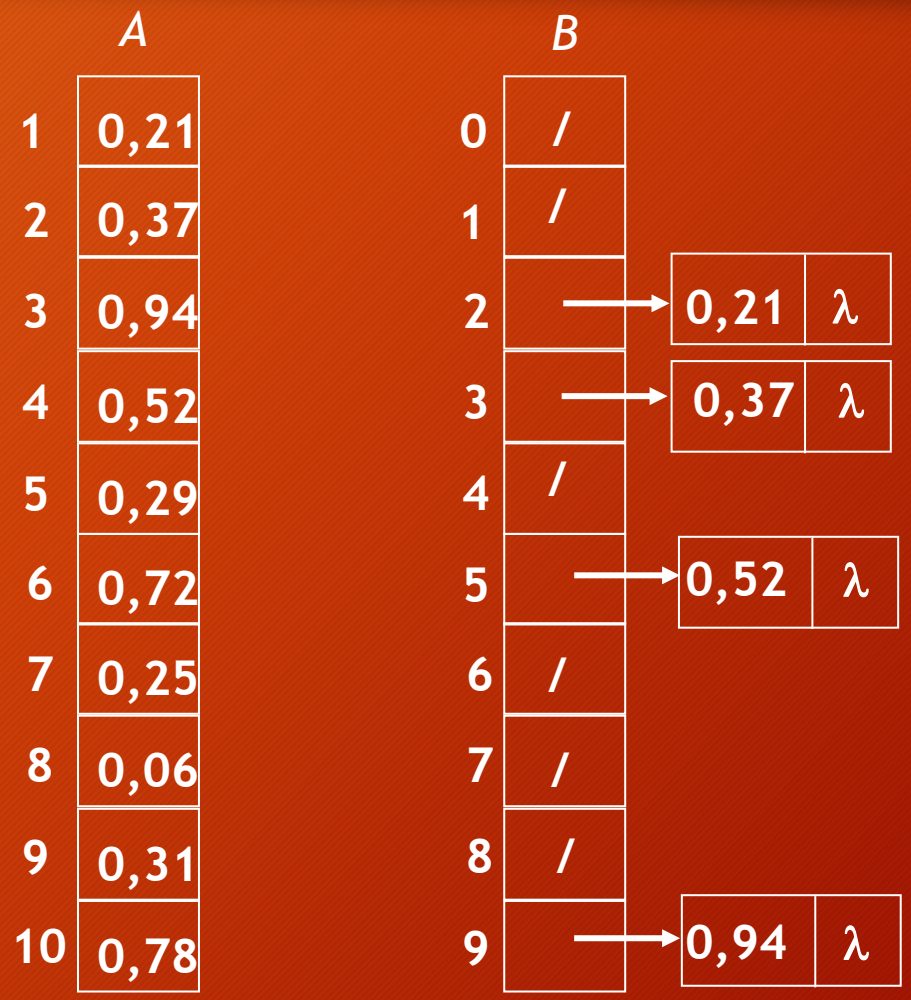
```
ordena_balde(A)

n ← comprimento_A

para i ← 1 até n faça
    inserir A[i] na lista B[⌊nA[i]⌋]

para i ← 0 até n-1 faça
    ordenar lista B[i] com Ordenação por Inserção

concatenar as listas B[0], ..., B[n-1] em ordem
```



Ordenação por Baldes (Bucket_sort)

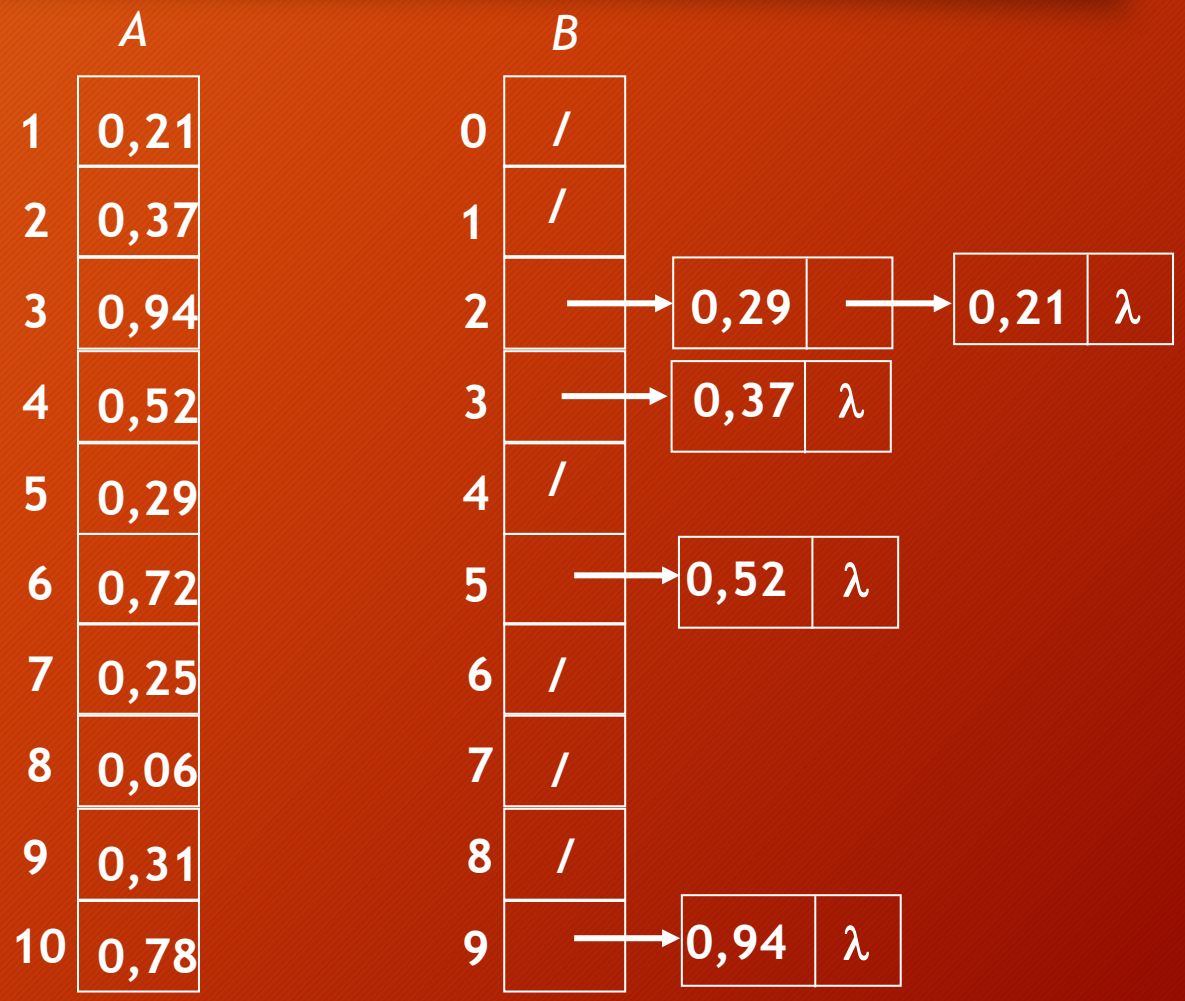
```
ordena_balde(A)

n ← comprimento_A

para i ← 1 até n faça
    inserir A[i] na lista B[⌊nA[i]⌋]

para i ← 0 até n-1 faça
    ordenar lista B[i] com Ordenação por Inserção

concatenar as listas B[0], ..., B[n-1] em ordem
```



Ordenação por Baldes (Bucket_sort)

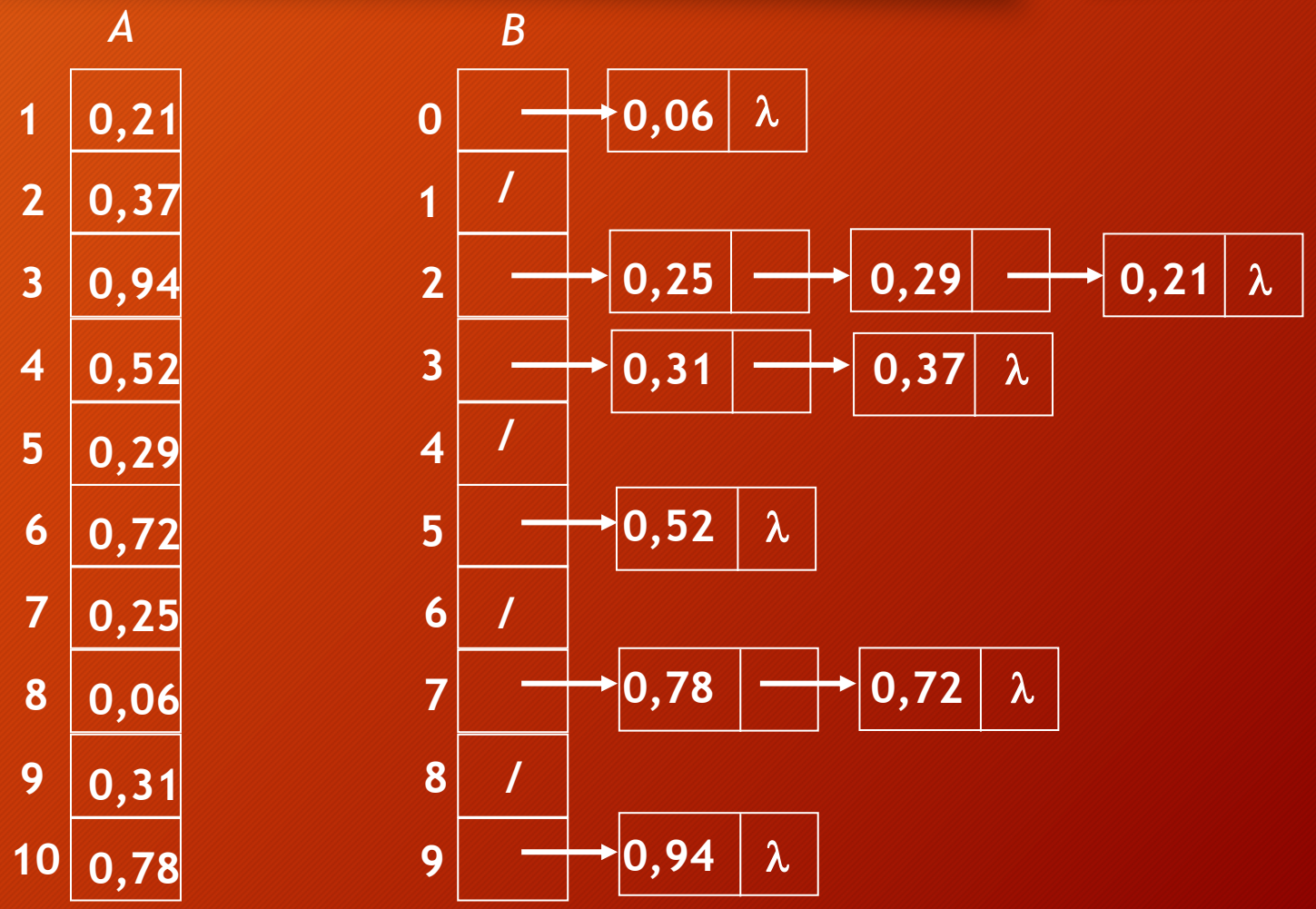
```
ordena_balde(A)

n ← comprimento_A

para i ← 1 até n faça
    inserir A[i] na lista B[⌊nA[i]⌋]

para i ← 0 até n-1 faça
    ordenar lista B[i] com Ordenação por Inserção

concatenar as listas B[0], ..., B[n-1] em ordem
```



Ordenação por Baldes (Bucket_sort)

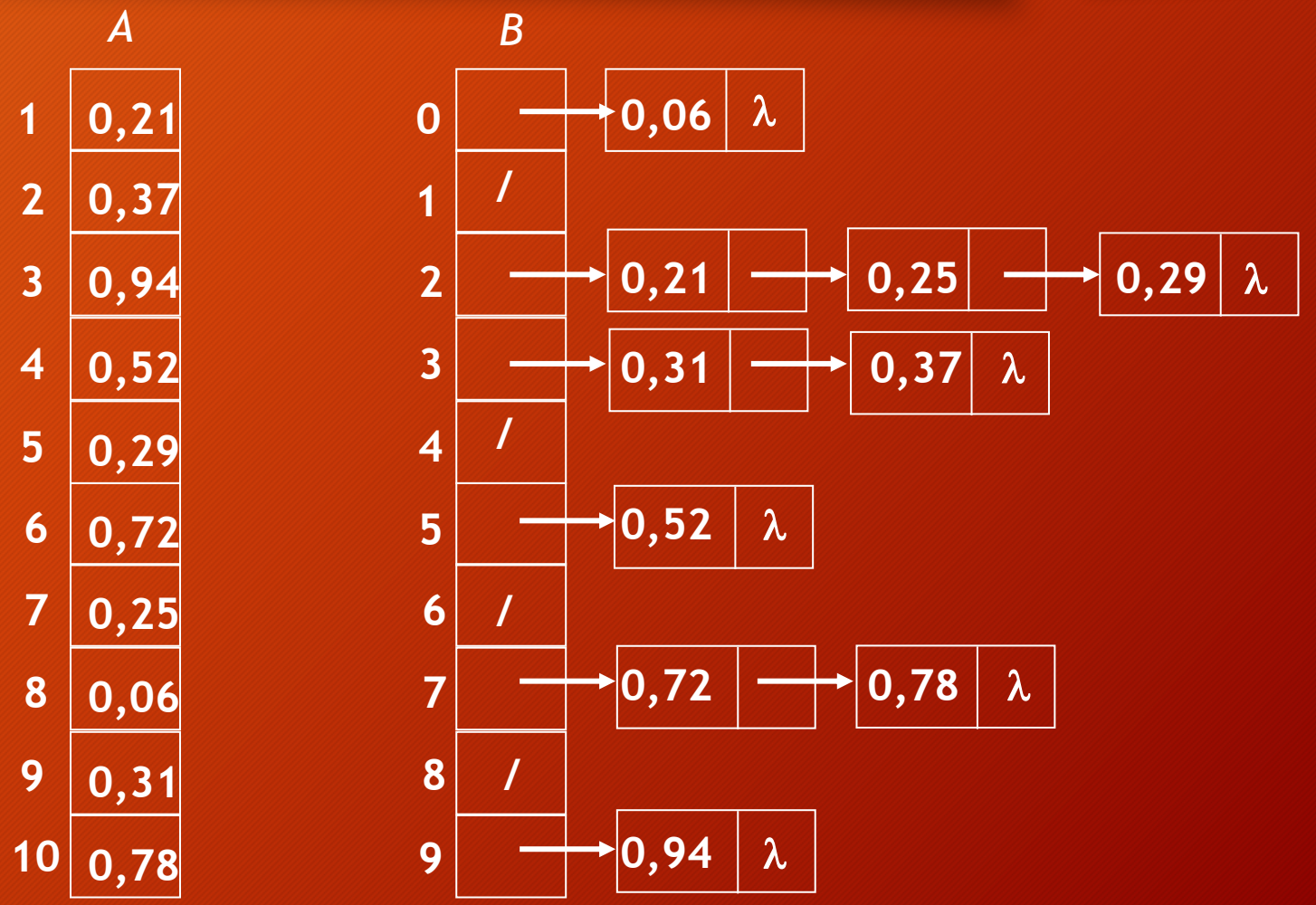
```
ordena_balde(A)

n ← comprimento_A

para i ← 1 até n faça
    inserir A[i] na lista B[⌊nA[i]⌋]

para i ← 0 até n-1 faça
    ordenar lista B[i] com Ordenação por Inserção

concatenar as listas B[0], ..., B[n-1] em ordem
```



Ordenação por Baldes (Bucket_sort)

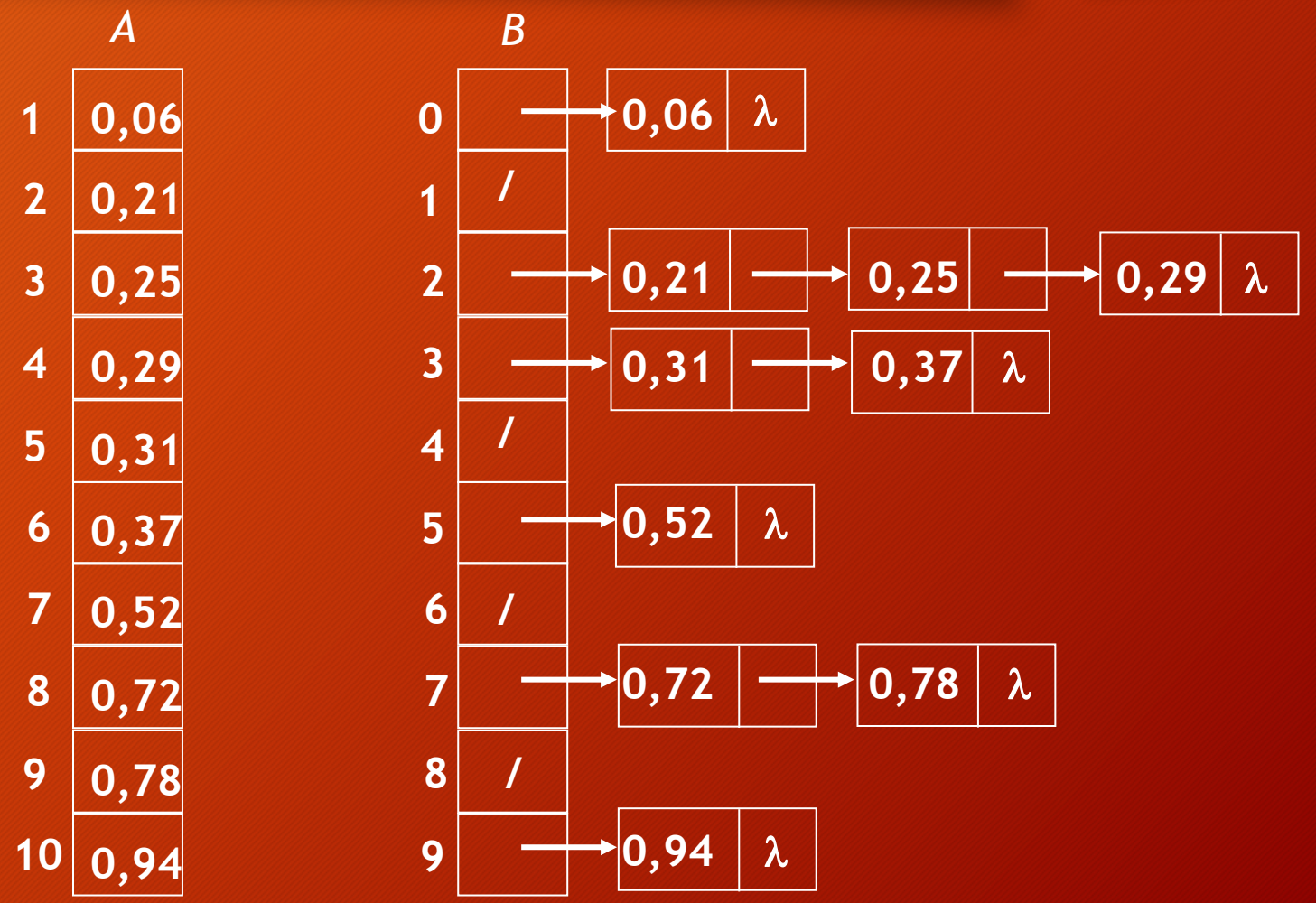
```
ordena_balde(A)

n ← comprimento_A

para i ← 1 até n faça
    inserir A[i] na lista B[⌊nA[i]⌋]

para i ← 0 até n-1 faça
    ordenar lista B[i] com Ordenação por Inserção

concatenar as listas B[0], ..., B[n-1] em ordem
```



Ordenação por Baldes - Complexidade

ordena_balde(*A*)

n ← *comprimento_A*

para *i* ← 1 até *n* faça
    inserir *A*[*i*] na lista *B*[⌊*nA*[*i*]⌋] θ(1)

} θ(*n*)

para *i* ← 0 até *n*-1 faça
    ordenar lista *B*[i] com Ordenação por Inserção

concatenar as listas *B*[0], ..., *B*[*n*-1] em ordem

Ordenação por Baldes - Complexidade

ordena_balde(*A*)

n ← *comprimento_A*

para *i* ← 1 até *n* faça
    inserir *A*[*i*] na lista *B*[⌊*nA*[*i*]⌋] θ(1) } θ(*n*)

para *i* ← 0 até *n*-1 faça
    ordenar lista *B*[i] com Ordenação por Inserção

concatenar as listas *B*[0], ..., *B*[*n*-1] em ordem θ(*n*)

Ordenação por Baldes - Complexidade

ordena_balde(*A*)

n ← *comprimento_A*

para *i* ← 1 até *n* faça
    inserir *A*[*i*] na lista *B*[⌊*nA*[*i*]⌋] θ(1)

} θ(*n*)

para *i* ← 0 até *n*-1 faça
    ordenar lista *B*[i] com Ordenação por Inserção → $O(n^2)$

} ?

concatenar as listas *B*[0], ..., *B*[*n*-1] em ordem θ(*n*)

Ordenação por Baldes - Complexidade

ordena_balde(*A*)

n ← *comprimento_A*

para *i* ← 1 até *n* faça
    inserir *A*[*i*] na lista *B*[$\lfloor nA[i] \rfloor$] **θ(1)**

θ(*n*)

para *i* ← 0 até *n*-1 faça
    ordenar lista *B*[i] com Ordenação por Inserção → **θ(n^2)**

?

concatenar as listas *B*[0], ..., *B*[*n*-1] em ordem → **θ(*n*)**

Ordenação por Baldes - Complexidade

Para analisar o tempo das chamadas para Ordenação por Inserção, considere n_i a variável aleatória que denota o número de elementos no balde $B[i]$.

O tempo do método é dado por $T(n) = \theta(n) + \sum_{i=0}^{n-1} O\left(n_i^2\right)$

Ordenação por Baldes - Complexidade

Para analisar o tempo das chamadas para Ordenação por Inserção, considere n_i a variável aleatória que denota o número de elementos no balde $B[i]$.

O tempo do método é dado por $T(n) = \theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$

$E[T(n)] = E[\theta(n) + \sum_{i=0}^{n-1} O(n_i^2)]$

Ordenação por Baldes - Complexidade

Para analisar o tempo das chamadas para Ordenação por Inserção, considere n_i a variável aleatória que denota o número de elementos no balde $B[i]$.

O tempo do método é dado por $T(n) = \theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$

$E[T(n)] = E[\theta(n) + \sum_{i=0}^{n-1} O(n_i^2)]$

$= \theta(n) + \sum_{i=0}^{n-1} E\ [O(n_i^2)]$

Ordenação por Baldes - Complexidade

Para analisar o tempo das chamadas para Ordenação por Inserção, considere n_i a variável aleatória que denota o número de elementos no balde $B[i]$.

O tempo do método é dado por $T(n) = \theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$

$$E[T(n)] = E[\theta(n) + \sum_{i=0}^{n-1} O(n_i^2)]$$

$$= \theta(n) + \sum_{i=0}^{n-1} E\ [O(n_i^2)]$$

$$= \theta(n) + \sum_{i=0}^{n-1} O(E\ [n_i^2])$$

Ordenação por Baldes - Complexidade

$E\left[n_i^2\right] = \text{Var}[n_i] + E^2[n_i]$

$n_i = k$ (distribuição binomial $b(k; n, p)$

Ordenação por Baldes - Complexidade

$E\left[n_i^2\right] = \text{Var}[n_i] + E^2[n_i]$

$n_i = k$ (distribuição binomial $b(k; n, p)$

$E[n_i] = np = 1 \longrightarrow p = 1/n$ (distribuição uniforme)

$\text{Var}[n_i] = np(1-p) = 1 - 1/n$

Ordenação por Baldes - Complexidade

$E\left[n_i^2\right] = \text{Var}[n_i] + E^2[n_i]$

$n_i = k$ (distribuição binomial $b(k; n, p)$)

$E[n_i] = np = 1 \longrightarrow p = 1/n$ (distribuição uniforme)

$\text{Var}[n_i] = np(1-p) = 1 - 1/n$

$E\left[n_i^2\right] = 1 - 1/n + 1^2$

$= 2 - 1/n$

O tempo do método é, portanto, $\theta(n) + nO(2-1/n) = \theta(n)$